3.1 使用Header接收请求头参数

需求场景

假设现在我们有个图书网站，需要客户端在请求头上携带有效的token，才可以获取图书数据，那FastAPI该如何获取客户端传过来的请求头参数呢？

FastAPI的解决方式

示例1：FastAPI获取 指定请求头的值（比如，请求头的key是 `token`）

```
from fastapi import FastAPI, Header

app = FastAPI()
books = {i: f"图书{i}" for i in range(10)}


@app.get("/books/{id}")
def get_book_by_id(id: int, token: str = Header()):
    if token == "this-is-token":
        return books.get(id)
    else:
        return "invalid token"
```

> 补充1：就和使用Path()，Query()类似，Header()也具有基本的请求头参数校验和API文档设置的功能
>
> 补充2：如果定义的参数是请求头参数，则必须使用Header()，否则FastAPI默认会把这个参数识别为查询参数

示例2：请求头参数可以有多个值，使用Header()默认只获取第一个值，使用`typing.List[]`来获取多有的值

```
import typing
from fastapi import FastAPI, Header


app = FastAPI()
books = {i: f"图书{i}" for i in range(10)}


@app.get("/books/{id}")
def get_book_by_id(id: int, token: typing.List[str] = Header()):
    return token
```

> 比如下面的请求，则返回：`["foo", "bar"]`
>
> ```
> curl -X 'GET' \
> 'http://127.0.0.1:8000/books/2' \
> -H 'accept: application/json' \
> -H 'token: foo,bar'
> ```

示例3：使用 `convert_underscores` 设置转换连接符，默认为 True

```
from fastapi import FastAPI, Header

app = FastAPI()
books = {i: f"图书{i}" for i in range(10)}


@app.get("/books/{id}")
def get_book_by_id(id: int, x_token: str = Header()):
    if x_token == "this-is-token":
        return books.get(id)
    else:
        return "invalid token"
```

> 补充1：因为默认在请求头中参数一般使用 `-` 做连接符，比如：`x-token`， 但是Python中使用 `_`，所以这中间一般需要做一个转化。
>
> 补充2：请求头也可以使用`_`，但是一般会使用 `-`

3.2 使用请求体提交数据

需求场景

向服务请提交数据，可以通过路径参数，查询参数，也可以在请求体中携带数据（推荐）

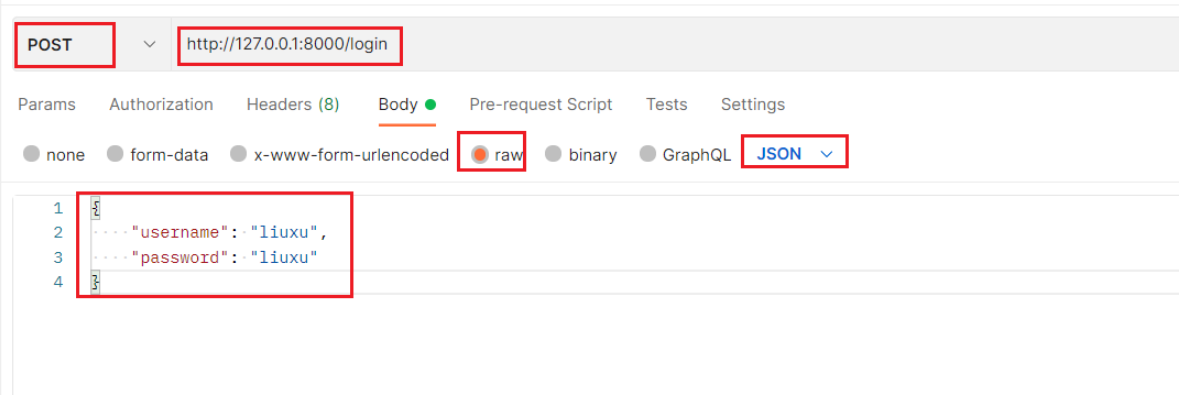
推荐使用请求体向服务器提交数据的原因：

- 路径参数和查询参数携带的数据量有限
- 路径参数和查询参数携带数据不安全

那我们该如何使用请求体向服务端提交数据呢？一般有两种方式：

- 使用POST请求，在请求体中使用JSON格式的数据；PUT/PATCH等也可以在请求体中使用JSON格式数据
- 使用POST请求，在请求体中使用form-data格式的数据，提交数据（即前端的form表单，第6章详细介绍）
- 上述两种方式，在一个接口中不能同时使用

示例1：使用POST请求，提交JSON格式的数据（使用postman）



3.3 使用Pydantic接收请求体数据

需求场景

知道了客户端需要通过请求体给服务端传数据，那在FastAPI中该如何获取客户端端传过来的请求体数据呢？

- 本节课我们先来看下FastAPI中如何获取JSON格式的请求体数据
- 在第6章我们再详细介绍如何获取表单数据

比如，现在需要一个登录接口`/login`，POST方式，请求体为，

```
{
    "username": "liuxu",
    "password": "liuxu"
}
```

FastAPI使用Pydantic来接收请求体数据

上面这种情况下，FastAPI分三步来实现获取请求体参数：

- 第一步：继承Pydantic的 `BaseModel`，定义模型类 `User` ，在类中定义两个和请求体字段同名的类属性并做类型提示
- 第二步：在路径函数中定义形参 `user`，并设置类型为 `User`
- 第三步：通过对象`user`属性`username`和`password`获取请求体字段的值

```
from fastapi import FastAPI
from pydantic import BaseModel
```

```python
app = FastAPI()


class User(BaseModel):
    username: str
    password: str


@app.post("/login")
def login(user: User):
    return {
        "username": user.username,
        "password": user.password
    }
```

这样做可以帮我们获取请求体参数并赋值到参数`user`，同时背后会做这些事情，

- （1）按照json格式来读取请求体数据
- （2）类型转换和类型校验
- （3）把请求体中每个字段的值赋值到模型类对象的每个属性
- （4）API文档

结论：

- 如果请求体参数是JSON格式，就通过继承Pydantic的 `BaseModel` 的方式来获取数据
- `BaseModel`的子类中定义的类属性是用来匹配请求体对应的字段值，所以名字包保持一致，字段的顺序没有要求
- 只要路径函数式中形参的类型是`BaseModel`的子类，则FastAPI会默认会把请求体数据保存在这个变量中
- 定义的模型类的属性，可以有默认值

3.4 多个请求体参数

需求场景

有的时候，客户端向后端提交的请求体数据可能是如下格式，此时该如何获取数据呢？

```
{
    "user": {
        "username": "liuxu",
        "password": "liuxu"
    },
    "item": {
        "name": "Foo",
        "description": "The pretender",
        "price": 42.0,
        "tax": 3.2
    },
}
```

FastAPI的解决方式

向上面这种JSON格式的请求体数据，它有两个外层字段：`user`和 `item`，此时我们需要定义两个模型类来分别接收

```
import typing
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class User(BaseModel):
    username: str = "aaaa"
    password: str


class Item(BaseModel):
    name: str
    description: typing.Optional[str] = None
    price: float
    tax: typing.Optional[float] = None


@app.post("/login")
def login(user: User, item: Item):
    return {
        "user": {
            "username": user.username,
            "password": user.password,
        },
        "item": {
            "name": item.name,
            "description": item.description,
            "price": item.price,
            "tax": item.tax,
        }
    }
```

> 补充：如果你仅需要获取请求体中的item字段的值，则可以直接使用：`def login(item: Item):`

结论：

- 路径函数中的形参`user`的类型是`User`用来接收请求体数据中key为`user`的数据
- 路径函数中的形参`item`的类型是`Item`用来接收请求体数据中key为`item`的数据
- 模型类中的每个类属性要个严格对应请求体中的每个字段
- 模型类的每个类属性的类型提示可以做类型转换和类型校验，默认值等等

3.5 使用Body接收请求体数据

需求场景

需求1：客户端的请求体数据格式如下，FastAPI该如何获取？

```
{
    "item": {
        "name": "Foo",
        "description": "The pretender",
        "price": 42.0,
        "tax": 3.2
    },
    "user": {
        "username": "liuxu",
        "password": "liuxu"
    },
    "importance": 5
}
```

需求2：请求体数据格式如下，该如何获取？

```
{
    "item": {
        "name": "Foo",
        "description": "The pretender",
        "price": 42.0,
        "tax": 3.2
    }
}
```

FastAPI的解决方式

上面的两种情况，直接使用继承Pydantic的BaseModel定义模型类的方式行不通，需要借助`Body()`函数来帮忙

示例1：使用Body()获取请求体中单一字段变量，比如需求1中的 `"importance": 5`

```python
import typing
from fastapi import FastAPI, Body
from pydantic import BaseModel

app = FastAPI()


class User(BaseModel):
    username: int
    password: str


class Item(BaseModel):
    name: str
    description: typing.Optional[str] = None
    price: float
    tax: typing.Optional[float] = None


@app.post("/login")
def login(user: User, item: Item, importance: int = Body()):    # 定义和请求体字段同名的形参并使用Body()
    return importance
```

> 补充1：Body()函数的使用和前面的Path()，Query()，Header()基本一致
>
> 补充2：如果你仅仅需要从请求体中获取importance，其他字段不需要，可直接：`def login(importance: int = Body()):`

示例2：使用Body()获取内嵌单一字段，比如需求2

```python
from fastapi import FastAPI, Body
from pydantic import BaseModel

app = FastAPI()


class User(BaseModel):
    username: int
    password: str


@app.post("/login")
def login(user: User = Body(embed=True)):
    return {
        "username": user.username,
        "password": user.password,
    }
```

3.6 请求体使用Body校验

结论：

- Body和前面接收的 Path、Query、Header一样，都可以做数值校验和字符串的基本校验
- 只不过Body校验的是请求体中的单一数据，比如请求体中只有一个数字或者一个字符串才可以
- 除此之外，还可以设置请求体的类型，默认是`application/json`，即JSON格式

```
from fastapi import FastAPI, Body

app = FastAPI()


@app.post("/login")
def login(name: str = Body(min_length=3), age: int = Body(ge=18)):
    return {"name": name, "age": age}
```

3.7 模型类嵌套

需求场景

客户端传过来的请求体数据结构是复杂的嵌套结构，比如下面这种，此时后端该如何接收数据呢？

需求1：嵌套列表

```
{
    "name": "Foo",
    "description": "The pretender",
    "price": 42.0,
    "tax": 3.2,
    "tags": ["rock", "metal", "bar"]
}
```

需求2：嵌套模型类

```
{
    "name": "Foo",
    "description": "The pretender",
    "price": 42.0,
    "tax": 3.2,
    "image": {
        "url": "http://example.com/baz.jpg",
        "name": "The Foo live"
    }
}
```

需求3：嵌套列表中的模型类

```
{
    "name": "Foo",
    "description": "The pretender",
    "price": 42.0,
    "tax": 3.2,
    "images": [
        {
            "url": "http://example.com/baz.jpg",
            "name": "The Foo live"
        },
        {
            "url": "http://example.com/dave.jpg",
            "name": "The Baz"
        }
    ]
}
```

FastAPI的解决方式

示例1：使用 `typing.List[]` 解决嵌套列表（数组）

```
from typing import List, Optional

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    description: Optional[str] = None
    price: float
    tax: Optional[float] = None
    tags: List[str] = []


@app.post("/item")
def create_item(item: Item):
    return item
```

> 补充1：因为 `tags: List[str]`，所以tags中的每个元素都会被转化成字符串类型
>
> 补充2：如果：`tags: typing.Set[str] = set()`，则请求体中的tags数组中的元素会被去重

示例2：使用 模型类 解决模型类嵌套

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
```

```
class Image(BaseModel):
    url: str
    name: str


class Item(BaseModel):
    name: str
    price: float
    image: Image            # 这个字段的类型是Image模型类，则这个字段的值应该是一个包含
url和name两个字段的对象


@app.post("/item")
def create_item(item: Item):
    return item
```

> 补充：URL字段的类型除了使用简单的str，还可以使用更加准确的 `HttpUrl`，好处是有了URL格式的校验功能
>
> ```
> from pydantic import BaseModel, HttpUrl
>
> class Image(BaseModel):
> url: HttpUrl
> name: str
> ```

示例3：解决 嵌套列表中的模型类

```
from typing import List, Optional
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class Image(BaseModel):
    url: str
    name: str


class Item(BaseModel):
    name: str
    price: float
    images: Optional[List[Image]] = None


@app.post("/item")
def create_item(item: Item):
    return item
```

3.8 请求体字段使用Filed校验

需求场景

请求体的每个字段可以单独做校验吗，比如：

- `name`的长度最少是3，
- `price`不少于10
- `tags字段限制元素不重复，限制元素的个数等等

```
{
    "name": "Foo",
    "description": "The pretender",
    "price": 42.0,
    "tax": 3.2,
    "tags": ["rock", "metal", "bar"]
}
```

FastAPI的解决方式

示例1：使用 `Field`来校验每个请求体字段，注意，`Field`不是从FastAPI导入的，而是直接从Pydantic导入的

```
from typing import Optional, List

from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()


class Item(BaseModel):
    name: str = Field(min_length=5)
    description: Optional[str] = Field(default=None, max_length=300)
    price: float = Field(gt=0, multiple_of=2)
    tax: List[str] = Field(min_items=2, max_items=5, unique_items=True)


@app.post("/item")
def create_item(item: Item):
    return item
```

校验参数总结：

```
gt/ge/lt/le:                    数值大小校验(float)
min_length/max_length            字符串长度校验(int)
min_items/max_items              集合元素数量校验（int）
unique_items                    集合元素是否可以重复(布尔值)
multiple_of                     限制值的某个数字的倍数（float）

max_digits                      最长多少位数字【bug】
decimal_places                  最多几位小数【bug】
```

示例2：使用默认工厂 `default_factory`

```python
from typing import Optional
import random
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()


class Item(BaseModel):
    name: str = Field(min_length=5)
    description: Optional[str] = Field(default=None, max_length=300)
    discount: int = Field(default_factory=lambda: random.randint(1, 9))
    # 请求体中如果没有discount字段，则随机出来一个1-9之间的数字当做discount的值

@app.post("/item")
def create_item(item: Item):
    return item
```

3.9 总结获取请求体数据的3种方式

使用Pydantic

- 定义模型类，和请求体中的字段保持一致，定义模型类的属性并做类型提示（支持类型校验和类型转换）

使用Body()

- 有个特殊的请求体格式，使用Body()显示获取，比如：单一字段的值，内嵌字段的值

使用Python的字典类型和列表类型

- 定义在路径函数内的特殊字段会默认被是被为请求体参数，比如：列表，集合，元组，字典（容器型数据）

```
import typing
from fastapi import FastAPI

app = FastAPI()


@app.post("/item")
def create_item(item: typing.Dict[str, str]):
    return item
```

- 普通的类也可以哦，需要借助 `@dataclasses.dataclass`

```
import dataclasses
from fastapi import FastAPI


app = FastAPI()


@dataclasses.dataclass
class User:
    username: str
    password: str


@app.post("/item")
def create_item(user: User):
    return user
```

3.10 给请求体提供范例

需求场景

希望在API文档中为api的请求体参数提供范例，方便前端阅读API文档

FastAPI的解决方式

FastAPI提供了多种为请求体提供范例的方式

示例1：在模型类中定义 `Config` 类

```
from typing import Union

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
```

```
class Item(BaseModel):
    name: str
    description: Union[str, None] = None
    price: float
    tax: Union[float, None] = None

    class Config:
        schema_extra = {
            "example": {
                "name": "Foo",
                "description": "A very nice Item",
                "price": 35.4,
                "tax": 3.2,
            }
        }


@app.post("/item")
def create_item(item: Item):
    return item
```

示例2：使用 `Field(example="xxx")` 为每个字段提供例子

```
from typing import Union

from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()


class Item(BaseModel):
    name: str = Field(example="Foo")
    description: Union[str, None] = Field(default=None, example="A very nice
Item")
    price: float = Field(example=35.4)
    tax: Union[float, None] = Field(default=None, example=3.2)


@app.post("/item")
def create_item(item: Item):
    return item
```

示例3：使用 `Body(example="一个例子", examples="多组例子")`，这种方式也适用：`Path\Query\Header` 等等

```
from typing import Union

from fastapi import Body, FastAPI
from pydantic import BaseModel
```

```python
app = FastAPI()


class Item(BaseModel):
    name: str
    description: Union[str, None] = None
    price: float
    tax: Union[float, None] = None


@app.post("/item")
def create_item(
    *,
    item: Item = Body(
        examples={
            "normal": {
                "summary": "A normal example",
                "description": "A **normal** item works correctly.",
                "value": {
                    "name": "Foo",
                    "description": "A very nice Item",
                    "price": 35.4,
                    "tax": 3.2,
                },
            },
            "converted": {
                "summary": "An example with converted data",
                "description": "FastAPI can convert price `strings` to actual
`numbers` automatically",
                "value": {
                    "name": "Bar",
                    "price": "35.4",
                },
            },
            "invalid": {
                "summary": "Invalid data is rejected with an error",
                "value": {
                    "name": "Baz",
                    "price": "thirty five point four",
                },
            },
        },
    ),
):
    return item
```